

ASIGNACION INDIVIDUAL

ELOY

FastAPI / Backend / APIs / Autenticacion

Proyecto	Sistema Punto de Venta e Inventario Multi-Sede
Sprint	Sprint 3
Periodo	31 de agosto - 6 de septiembre
Rama real	Eloy-FAstAPI
PR destino	main

Documento listo para ejecutar y presentar como evidencia individual.

1. Objetivo y limites de responsabilidad

Eloy es responsable del dominio central de FastAPI: autenticacion, sesiones, permisos generales, usuarios y configuracion de la aplicacion. Su trabajo debe permitir que el resto de las APIs reutilice una identidad segura obtenida desde PostgreSQL.

Resultado obligatorio

- POST /auth/login consulta PostgreSQL y verifica bcrypt.
- GET /auth/me devuelve la identidad actual.
- POST /auth/logout elimina la sesion.
- La cookie firmada guarda solamente user_id.
- get_current_user vuelve a consultar PostgreSQL en cada solicitud.
- require_admin, require_admin_or_manager y require_authenticated distinguen permisos.
- CRUD de usuarios completo y disponible unicamente para ADMIN.
- app/main.py registra middleware, routers, templates, archivos estaticos y rutas HTML.

No debe modificar

- Modelos SQLAlchemy, Alembic o estructura PostgreSQL (Keyla).
- Routers/schemas de sucursales, productos, categorias e inventario (Xanic).
- HTML, CSS o JavaScript visual (Estefania y Kevin).

Regla de autoridad: el navegador nunca decide user_id, rol ni sucursal. FastAPI obtiene user_id de la sesion y carga rol/sucursal desde PostgreSQL.

2. Dependencias, dificultad y tiempo

Concepto	Detalle
Depende de	PR de Keyla: modelos, relaciones, Alembic y roles
Desbloquea	Xanic (permisos) y Estefania (login frontend)
Dificultad	Difícil
Tiempo	12-16 horas para un estudiante

3. Paso 0 - Preparar Git

```
git status
git switch main
git pull origin main
git switch Eloy-FastAPI
git merge main
```

Despues de que el PR de Keyla sea integrado:

```
git fetch origin
git merge origin/main
```

Si git status muestra cambios que deben conservarse, revisar git diff y crear un commit especifico o usar git stash. No usar git add . y no subir .env, .venv, .idea, __pycache__ ni archivos .pyc.

4. Archivos propios

Accion	Archivos
Modificar	requirements.txt; app/main.py; app/routers/auth.py; app/routers/users.py; app/schemas/user.py
Crear	.env.example; app/dependencies/__init__.py; app/dependencies/auth.py
No tocar	app/models/*; migrations/*; routers/schemas de Xanic; app/templates/*; app/static/*

5. Configuracion y variables

requirements.txt - agregar al final:

```
alembic==1.16.5
itsdangerous==2.2.0
```

.env.example - crear sin credenciales reales:

```
DATABASE_URL=postgresql+psycopg2://usuario:contrasena@localhost:5432/pi_net
SESSION_SECRET=cambiar-por-una-clave-larga-y-aleatoria
ADMIN_NAME=Administrador General
ADMIN_EMAIL=admin@example.com
ADMIN_PASSWORD=cambiar-esta-contrasena
```

SESSION_SECRET firma la cookie para detectar modificaciones. La cookie firmada no es una base de datos ni debe contener secretos; por eso solo guardara user_id.

6. Schemas de usuario

Conservar UserLogin compatible con el frontend y agregar AuthUser y UserUpdate en app/schemas/user.py:

```
class UserLogin(BaseModel):
    email: str
    password: str

class AuthUser(BaseModel):
    id: int
    nombre: str
    correo: str
    rol: str
    sucursal_id: int | None

class UserUpdate(BaseModel):
    nombre: str
    correo: str
    password: str | None = None
    rol_id: int
    sucursal_id: int | None = None
    activo: bool = True
```

UserUpdate permite editar un usuario sin forzar un cambio de contrasena. UserResponse nunca incluye password_hash.

7. Dependencias de autenticacion

Crear app/dependencies/__init__.py vacio y app/dependencies/auth.py con:

```
from collections.abc import Callable

from fastapi import Depends, HTTPException, Request, status
from sqlalchemy.orm import Session

from app.database import get_db
from app.models.user import User

def get_current_user(
    request: Request,
    db: Session = Depends(get_db),
) -> User:
    user_id = request.session.get("user_id")

    if user_id is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
```

```

        detail="Debes iniciar sesion",
    )

    user = (
        db.query(User)
        .filter(User.id == user_id, User.activo.is_(True))
        .first()
    )

    if user is None:
        request.session.clear()
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Sesion invalida o usuario inactivo",
        )

    return user

def require_roles(*allowed_roles: str) -> Callable:
    def check_role(
        current_user: User = Depends(get_current_user),
    ) -> User:
        if current_user.rol is None:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="El usuario no tiene un rol valido",
            )

        if current_user.rol.nombre not in allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="No tienes permiso para esta operacion",
            )

        return current_user

    return check_role

require_admin = require_roles("ADMIN")
require_admin_or_manager = require_roles("ADMIN", "GERENTE")
require_authenticated = require_roles("ADMIN", "GERENTE", "CAJERO")

```

Depends indica que FastAPI debe resolver primero otra funcion. 401 significa que no hay una identidad valida; 403 significa que el usuario si esta identificado, pero no tiene permiso.

8. Login real, identidad y logout

Reemplazar el endpoint falso de app/routers/auth.py por:

```

import bcrypt
from fastapi import APIRouter, Depends, HTTPException, Request
from sqlalchemy.orm import Session

from app.database import get_db
from app.dependencies.auth import get_current_user
from app.models.user import User
from app.schemas.user import AuthUser, UserLogin

router = APIRouter(prefix="/auth", tags=["Authentication"])

def to_auth_user(user: User) -> AuthUser:
    return AuthUser(
        id=user.id,
        nombre=user.nombre,
        correo=user.correo,
        rol=user.rol.nombre,
        sucursal_id=user.sucursal_id,
    )

@router.post("/login", response_model=AuthUser)
def login(
    data: UserLogin,
    request: Request,
    db: Session = Depends(get_db),
):
    user = db.query(User).filter(User.correo == data.email).first()

    if user is None:

```

```

        raise HTTPException(
            status_code=401,
            detail="Correo o contraseña incorrectos",
        )

    if not user.activo:
        raise HTTPException(status_code=401, detail="Usuario inactivo")

    valid_password = bcrypt.checkpw(
        data.password.encode("utf-8"),
        user.password_hash.encode("utf-8"),
    )

    if not valid_password:
        raise HTTPException(
            status_code=401,
            detail="Correo o contraseña incorrectos",
        )

    if user.rol is None:
        raise HTTPException(
            status_code=403,
            detail="El usuario no tiene un rol válido",
        )

    request.session.clear()
    request.session["user_id"] = user.id
    return to_auth_user(user)

@router.get("/me", response_model=AuthUser)
def me(current_user: User = Depends(get_current_user)):
    return to_auth_user(current_user)

@router.post("/logout")
def logout(request: Request):
    request.session.clear()
    return {"message": "Sesión cerrada"}

```

`bcrypt.checkpw` compara la contraseña recibida con el hash almacenado. No descifra el hash. La respuesta solo contiene identidad útil para la interfaz.

9. CRUD de usuarios

Todos los endpoints de `app/routers/users.py` deben depender de `require_admin`. Agregar GET `/usuarios/{usuario_id}` y validar rol, sucursal, correo y contraseña.

```

def validate_role_and_branch(
    rol_id: int,
    sucursal_id: int | None,
    db: Session,
) -> Role:
    role = db.query(Role).filter(Role.id == rol_id).first()
    if role is None:
        raise HTTPException(status_code=404, detail="Rol no encontrado")

    if role.nombre == "ADMIN":
        if sucursal_id is not None:
            raise HTTPException(
                status_code=422,
                detail="El administrador no debe tener sucursal",
            )
        return role

    if sucursal_id is None:
        raise HTTPException(
            status_code=422,
            detail="Gerentes y cajeros deben tener sucursal",
        )

    branch = (
        db.query(Sucursal)
        .filter(Sucursal.id == sucursal_id, Sucursal.activo.is_(True))
        .first()
    )
    if branch is None:
        raise HTTPException(
            status_code=404,
            detail="Sucursal no encontrada o inactiva",
        )

    return role

```

Antes de crear:

```
existing_user = db.query(User).filter(User.correo == usuario.correo).first()
if existing_user:
    raise HTTPException(
        status_code=409,
        detail="El correo ya esta registrado",
    )
```

Al actualizar, cambiar password_hash solamente si UserUpdate.password tiene valor. DELETE sigue siendo desactivacion logica: usuario_db.activo = False. Si ocurre IntegrityError, ejecutar db.rollback() y devolver 409.

10. Configuracion exclusiva de app/main.py

Eloy es el unico propietario. Debe registrar los routers de Xanic, pero no editar su contenido ni disenar las vistas.

```
import os

from fastapi import FastAPI, Request
from fastapi.responses import RedirectResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from starlette.middleware.sessions import SessionMiddleware

from app.models import branch, category, inventory, product, user # noqa: F401
from app.routers import (
    auth,
    branches,
    categories,
    inventory,
    products,
    users,
)

templates = Jinja2Templates(directory="app/templates")
app = FastAPI(title="Punto de Venta Multi-Sede")

session_secret = os.getenv("SESSION_SECRET")
if not session_secret:
    raise ValueError("No se encontro SESSION_SECRET")

app.add_middleware(
    SessionMiddleware,
    secret_key=session_secret,
    same_site="lax",
    https_only=False,
)

app.mount("/static", StaticFiles(directory="app/static"), name="static")

app.include_router(auth.router)
app.include_router(users.router)
app.include_router(branches.router)
app.include_router(categories.router)
app.include_router(products.router)
app.include_router(inventory.router)

@app.get("/")
def root():
    return RedirectResponse(url="/login", status_code=302)

@app.get("/login")
def login_page(request: Request):
    return templates.TemplateResponse(request=request, name="login.html")

@app.get("/dashboard")
def dashboard_page(request: Request):
    return templates.TemplateResponse(request=request, name="dashboard.html")

@app.get("/sucursales-ui")
def branches_page(request: Request):
    return templates.TemplateResponse(request=request, name="branches.html")

@app.get("/productos-ui")
def products_page(request: Request):
    return templates.TemplateResponse(request=request, name="products.html")
```

```
@app.get("/inventario-ui")
def inventory_page(request: Request):
    return templates.TemplateResponse(request=request, name="inventory.html")
```

Durante desarrollo local `https_only` queda `False`. Antes de produccion debe cambiarse a `True`. Las rutas HTML solamente entregan archivos; la seguridad real esta en los endpoints API.

11. Pruebas individuales

```
python -m pip install -r requirements.txt
alembic upgrade head
python -m scripts.seed_admin
uvicorn app.main:app --reload
```

Abrir `http://127.0.0.1:8000/docs` y ejecutar:

Prueba	Esperado
POST /auth/login con admin correcto	200; identidad sin password_hash
POST /auth/login con password incorrecto	401
GET /auth/me despues del login	200; mismo usuario
POST /auth/logout y despues GET /auth/me	401
GET /usuarios/ como ADMIN	200
GET /usuarios/ como GERENTE o CAJERO	403
POST /usuarios/ con rol inexistente	404
POST /usuarios/ con correo duplicado	409
POST /usuarios/ gerente sin sucursal	422

Consulta de comprobacion en PostgreSQL

```
SELECT
    u.id,
    u.nombre,
    u.correo,
    u.activo,
    r.nombre AS rol,
    u.sucursal_id,
    u.password_hash
FROM usuarios u
JOIN roles r ON r.id = u.rol_id
ORDER BY u.id;
```

El hash debe existir en PostgreSQL, pero nunca aparecer en JSON de la API.

12. Errores frecuentes

Error	Que revisar
No module named itsdangerous	Reinstalar requirements.txt
SESSION_SECRET no encontrado	Crear .env desde .env.example
relation usuarios does not exist	Ejecutar alembic upgrade head
/auth/me devuelve 401 tras login	Usar mismo navegador; revisar cookie y credentials
Admin recibe 403	Comprobar que roles.nombre sea exactamente ADMIN
500 despues de IntegrityError	Ejecutar db.rollback() dentro del except
TemplateNotFound	Comprobar merge del frontend y nombres exactos

Error	Que revisar
CSS 404	Comprobar app.mount('/static', ...)

13. Guardar el trabajo y Pull Request

```
git status
git add requirements.txt .env.example app/main.py app/dependencies app/routers/auth.py app/routers/users.py app/schemas/user.py
git commit -m "feat(api): implementar autenticacion usuarios y configuracion FastAPI"
git push origin Eloy-FastAPI
```

Pull Request

- Rama origen: Eloy-FastAPI
- Rama destino: main
- Titulo: Sprint 3: autenticacion, usuarios y configuracion FastAPI
- Revisor recomendado: Keyla
- Debe indicar que depende del PR de modelos/migraciones de Keyla.
- Debe listar pruebas 200, 401, 403, 404, 409 y 422 realizadas.

14. Checklist de evidencia individual

- [] Captura del codigo de login y dependencias.
- [] Swagger: login correcto 200.
- [] Swagger: password incorrecto 401.
- [] Swagger: usuario autenticado sin permiso 403.
- [] Swagger: /auth/me y logout.
- [] CRUD de usuarios como ADMIN.
- [] Consulta PostgreSQL mostrando rol, sucursal y hash.
- [] Respuesta HTTP sin password_hash.
- [] Commit y push en Eloy-FastAPI.
- [] Pull Request hacia main.
- [] Revision realizada por otro integrante.
- [] Explicacion breve de lo implementado y probado.

15. Definicion de terminado para Eloy

- [] La aplicacion inicia con DATABASE_URL y SESSION_SECRET.
- [] Login consulta PostgreSQL y verifica bcrypt.
- [] Sesion guarda unicamente user_id.
- [] Rol y sucursal se cargan desde PostgreSQL.
- [] 401 y 403 estan correctamente diferenciados.
- [] Usuarios solo pueden administrarse con rol ADMIN.
- [] ADMIN puede no tener sucursal; GERENTE y CAJERO deben tenerla.
- [] Correo duplicado devuelve 409.
- [] password_hash nunca se devuelve.
- [] Todos los routers del Sprint 3 estan registrados.
- [] Templates y /static estan configurados.
- [] Pruebas documentadas y PR revisado.
- [] No se implementaron ventas, POS, detalles, tickets ni reportes en Sprint 3.